

WEEK 4 LEARNING GUIDE

BUILDING AN EVENT-DRIVEN SERVERLESS IMAGE PROCESSING SYSTEM

1. INTRODUCTION

This week focused on understanding **Event-Driven Architectures** and how AWS services can communicate asynchronously to build scalable serverless applications. While the previous weeks primarily dealt with Infrastructure as Code, serverless APIs, authentication, and database design, this week's learning introduced a different architectural approach where services react to events instead of direct client requests.

The project involved building a **Serverless Image Processing System** that automatically processed uploaded images and notified users upon completion. To achieve this, several new AWS services and concepts were introduced, including **Amazon S3 Event Notifications**, **Amazon Simple Queue Service (SQS)**, **Amazon Simple Notification Service (SNS)**, and **Lambda Event Source Mappings**. The week also expanded on Amazon Cognito by exploring advanced authentication features such as the Hosted UI, OAuth Authorization Code Flow, API Gateway JWT Authorizers, and Pre Token Generation triggers.

Rather than simply connecting AWS services together, the emphasis this week was on understanding how event-driven systems are designed, why asynchronous processing is important, and how loosely coupled architectures improve scalability and reliability. These concepts build directly upon the serverless foundations established during the previous weeks and provide a deeper understanding of designing cloud-native applications.

2. CORE CONCEPTS LEARNED

EVENT-DRIVEN ARCHITECTURE

One of the biggest architectural concepts introduced this week was **Event-Driven Architecture (EDA)**. Up until this point, every application developed during the internship followed a **request-response** model where each action was initiated by an HTTP request from the client. Although this approach works well for CRUD-based applications, it is not ideal for operations that require significant processing time or do not need to complete before responding to the user.

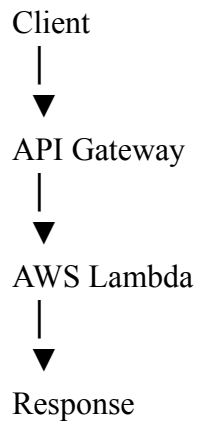
Event-Driven Architecture solves this problem by allowing services to communicate through **events** instead of direct requests. An event represents something that has already happened

within the system, such as a file upload, user login, or payment completion. Instead of directly invoking another service, the service generating the event simply publishes it, allowing other services to react independently.

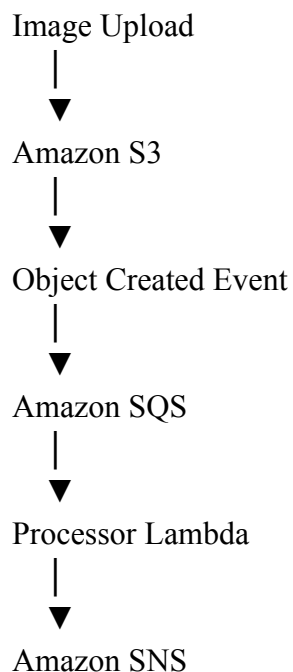
Request-Response vs Event-Driven Systems

The difference between these two approaches can be visualized below.

Request-Response



Event-Driven



In the request-response model, the client is responsible for triggering every action. In contrast, an event-driven system only requires the initial event. The remaining workflow is automatically coordinated by AWS services.

Why Event-Driven Architecture?

Modern cloud applications often consist of multiple independent services, each performing a specific task. If every service directly communicates with every other service, the application quickly becomes tightly coupled and difficult to maintain.

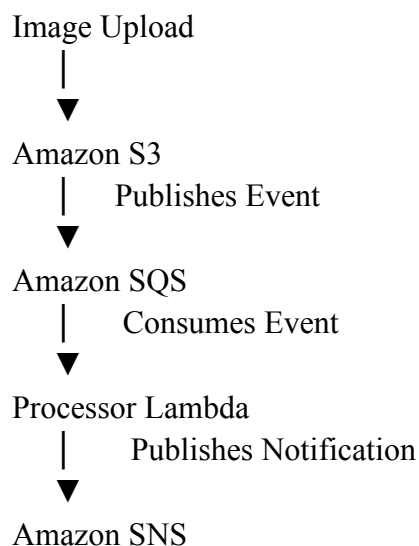
For example, consider an image processing application.

Without an event-driven approach, the upload service would need to:

- Upload the image
- Process the image
- Update the database
- Notify the user

This means a single service becomes responsible for several independent tasks.

Using events, these responsibilities are separated.



Each service performs only one responsibility, making the application easier to maintain and extend.

Advantages of Event-Driven Systems

During the implementation, several advantages of this architecture became evident.

- **Loose Coupling** – Services communicate through events rather than direct API calls, allowing them to evolve independently.
- **Scalability** – Individual components can scale according to workload without scaling the entire application.
- **Reliability** – Temporary failures in one service do not immediately affect the rest of the workflow.

- **Asynchronous Processing** – Long-running tasks execute in the background without increasing response times for users.

These characteristics make event-driven architectures particularly suitable for serverless applications.

Learning Through Implementation

Initially, I viewed AWS Lambda primarily as a backend service that handled API requests. This week introduced a different perspective where Lambda functions act as event processors, responding automatically whenever another AWS service generates an event.

This shifted my understanding from designing APIs to designing **workflows**, where each service performs a specific responsibility while remaining independent of the others.

Application in this Project

The image processing application was designed entirely around an event-driven workflow. Uploading an image to Amazon S3 generated an event that automatically triggered the processing pipeline. No additional API calls were required after the upload, demonstrating how loosely coupled services can coordinate complex workflows through events alone.

AMAZON S3 EVENT NOTIFICATIONS

Having understood the fundamentals of event-driven systems, the next concept introduced was **Amazon S3 Event Notifications**. This feature allows an S3 bucket to automatically notify other AWS services whenever specific events occur within the bucket.

Instead of continuously checking whether new files have been uploaded, S3 itself generates an event whenever a configured action takes place. This enables storage to become an active participant in an application's workflow rather than simply acting as a location for storing files.

Event Types

Amazon S3 can generate notifications for several different events, including:

- **Object Created** – Triggered whenever a new object is uploaded.
- **Object Removed** – Triggered when an object is deleted.
- **Object Restore** – Triggered when archived objects are restored.
- **Replication Events** – Triggered during cross-region replication.

For this project, only **Object Created** events were required since image uploads initiated the processing workflow.

Event Destinations

S3 Event Notifications can send events to multiple AWS services depending on the application's requirements.

- **AWS Lambda** – Execute processing immediately.
- **Amazon SQS** – Queue events for asynchronous processing.
- **Amazon SNS** – Broadcast events to multiple subscribers.

Choosing the appropriate destination depends on the application's architecture.

Example Configuration

The following Terraform configuration creates an event notification that sends every newly uploaded object to an SQS queue.

```
resource "aws_s3_bucket_notification" "image_events" {
  bucket = aws_s3_bucket.images.id
  queue {
    queue_arn = aws_sqs_queue.image_queue.arn
    events    = ["s3:ObjectCreated:*"]
  }
}
```

This demonstrates how Amazon S3 becomes an event producer. Whenever a new object is created, S3 automatically generates an event instead of waiting for another service to periodically check the bucket for changes.

Event Filtering

One useful feature of S3 Event Notifications is the ability to filter events using **prefixes** and **suffixes**.

For example:

uploads/

Only objects uploaded to the uploads/ folder trigger an event.

Similarly,

.jpg

ensures that only JPEG images generate notifications.

Filtering prevents unnecessary processing and allows multiple workflows to exist within the same bucket without interfering with one another.

Why Not Poll the Bucket?

Before learning about S3 Event Notifications, I assumed the application would periodically check the bucket for newly uploaded images.

Although this approach works, it introduces several disadvantages:

- Unnecessary API requests.
- Increased processing costs.
- Delayed detection of new uploads.
- Additional application logic.

Event Notifications eliminate these problems by allowing S3 to immediately notify other AWS services whenever an upload occurs.

Learning Through Implementation

Initially, the architecture considered invoking the processing Lambda directly after the upload completed. As the design evolved, it became clear that allowing S3 to generate events automatically resulted in a cleaner and more scalable solution.

Instead of relying on additional API calls from the frontend, the upload itself became the trigger for the entire processing pipeline.

Application in this Project

Once an image was uploaded successfully, the S3 bucket generated an **Object Created** event. Rather than invoking the processor directly, this event was forwarded to an Amazon SQS queue, allowing image uploads and image processing to remain completely independent.

AMAZON SIMPLE QUEUE SERVICE (AMAZON SQS)

Another major AWS service introduced this week was **Amazon Simple Queue Service (Amazon SQS)**. As the application moved towards an event-driven architecture, it became necessary to introduce a reliable way for different AWS services to exchange information without directly depending on one another.

Amazon SQS is a fully managed **message queuing service** that enables asynchronous communication between applications and AWS services. Instead of immediately invoking another service, producers place messages into a queue, where they remain until a consumer retrieves and processes them.

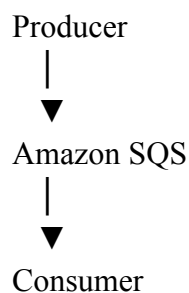
This decouples different parts of the application, making the overall system more scalable and fault tolerant.

Why AWS Provides SQS

In distributed systems, different services often operate at different speeds.

For example, uploading an image to Amazon S3 may take only a few seconds, while processing that image could take considerably longer. If the upload service waited for processing to complete, users would experience unnecessary delays and the two services would become tightly coupled.

Introducing a queue allows these services to operate independently.



The producer's responsibility ends once the message is successfully placed into the queue. The consumer can process it whenever resources become available.

Standard Queues vs FIFO Queues

Amazon SQS provides two types of queues.

Standard Queue

Standard Queues are designed for **maximum throughput** and are the default choice for most applications.

Characteristics include:

- Nearly unlimited throughput
- At-least-once message delivery
- Best-effort message ordering

Because messages can occasionally be delivered more than once or out of order, applications should be designed to handle duplicate processing.

FIFO Queue

FIFO (First-In First-Out) Queues guarantee that:

- Messages are processed in order.
- Duplicate messages are prevented.

However, these guarantees come with lower throughput compared to Standard Queues.

Since the image processing workflow did not depend on strict ordering, a Standard Queue was sufficient for this project.

Messages, Producers and Consumers

Every SQS queue consists of three basic components.

Message

A message contains the information that needs to be processed. In this project, the message included details about the uploaded S3 object.

Producer

The producer sends messages into the queue.

For this implementation:

- Amazon S3 acted as the producer.

Consumer

The consumer retrieves messages and performs the required processing.

In this project:

- The Processor Lambda acted as the consumer.

Example S3 Event

When Amazon S3 sends a message to the queue, it contains information about the uploaded object rather than the file itself.

```
{
  "Records": [{
    "s3": {
      "bucket": {
        "name": "image-bucket"
      },
    },
  ],
}
```



```
    "object": {  
      "key": "uploads/image.jpg"  
    }  
  }  
}  
}]  
}
```

The Processor Lambda reads this event, extracts the bucket name and object key, and then retrieves the image from Amazon S3 for further processing.

Visibility Timeout

One of the most important SQS concepts introduced this week was the **Visibility Timeout**.

When a consumer receives a message, SQS does **not** immediately remove it from the queue.

Instead, the message becomes temporarily invisible to other consumers.

Receive Message



Visibility Timeout



Processing



Delete Message

If processing completes successfully, the consumer deletes the message.

If processing fails before deletion, the visibility timeout expires and the message becomes available again for another processing attempt.

This provides automatic retry behaviour without requiring additional application logic.

Message Retention

SQS stores messages for a configurable retention period ranging from **1 minute to 14 days**.

If a message is not processed within this period, it is automatically removed from the queue.

This allows temporary consumer failures without risking immediate message loss.

Long Polling

Consumers can retrieve messages using either **Short Polling** or **Long Polling**.

With Short Polling, the consumer continuously checks the queue, even when no messages are available.

Long Polling waits until a message arrives before returning a response.

This reduces unnecessary API requests and lowers AWS costs.

Although this was handled automatically by Lambda during the implementation, understanding Long Polling helped explain how SQS improves efficiency.

Dead Letter Queues (DLQs)

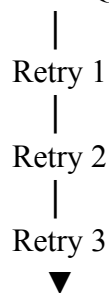
Not every message can be processed successfully.

Failures may occur due to:

- Corrupted input
- Invalid message format
- Missing resources
- Application bugs

Rather than retrying indefinitely, SQS allows failed messages to be moved into a **Dead Letter Queue (DLQ)** after a configurable number of attempts.

Main Queue



Dead Letter Queue

Dead Letter Queues prevent problematic messages from blocking normal processing while allowing developers to inspect failures later.

Although the project did not explicitly configure a DLQ, understanding this concept highlighted an important production best practice for building reliable event-driven systems.

At-Least-Once Delivery

Amazon SQS guarantees **at-least-once delivery**.

This means every message will eventually be delivered, but under certain conditions a message may be delivered more than once.

For this reason, consumers should ideally be designed to be **idempotent**, meaning processing the same message multiple times should not produce incorrect results.

This was an important distributed systems concept introduced during the week.

Learning Through Implementation

Initially, I assumed the uploaded image could directly invoke the Processor Lambda.

As the architecture evolved, introducing Amazon SQS between Amazon S3 and Lambda proved to be a better design. The queue separated image uploads from processing, introduced automatic retries, and made the overall system more resilient without significantly increasing complexity.

This demonstrated that adding an additional AWS service can sometimes simplify an application's architecture rather than complicate it.

Application in this Project

After an image was uploaded, Amazon S3 published an event to an SQS queue. The queue temporarily stored the event until the Processor Lambda retrieved and processed it. This decoupled image uploads from image processing while improving the scalability and reliability of the overall workflow.

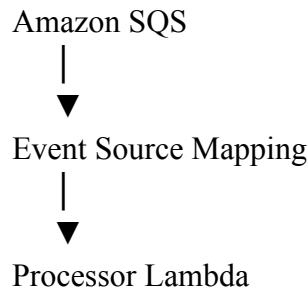
LAMBDA EVENT SOURCE MAPPINGS

In Week 2, Lambda functions were primarily invoked through **API Gateway**, where each execution corresponded to an incoming HTTP request.

This week introduced a different execution model known as a **Lambda Event Source Mapping**, where Lambda automatically polls another AWS service for work instead of waiting for client requests.

How Event Source Mappings Work

An Event Source Mapping acts as a bridge between an event source and a Lambda function.



AWS continuously monitors the queue on behalf of the application. Whenever new messages arrive, Lambda is automatically invoked without requiring any additional code.

Example Configuration

The following Terraform resource creates an Event Source Mapping between Amazon SQS and the processing Lambda.

```
resource "aws_lambda_event_source_mapping" "processor" {
  event_source_arn = aws_sqs_queue.image_queue.arn
  function_name    = aws_lambda_function.processor.arn
  batch_size       = 1
}
```

Instead of writing polling logic manually, AWS continuously monitors the queue and automatically invokes the Lambda function whenever new messages become available.

Automatic Polling

One advantage of Event Source Mappings is that developers do not need to write polling logic themselves.

Instead of repeatedly checking whether messages are available, AWS performs this operation automatically and invokes the Lambda function only when work exists.

This simplifies application development while reducing unnecessary API calls.

Batch Processing

Lambda can retrieve and process multiple messages during a single invocation.

The **Batch Size** determines how many messages are delivered together.

Smaller batch sizes reduce processing latency, while larger batches improve throughput by reducing the number of Lambda invocations.

Choosing an appropriate batch size depends on the application's workload.

Automatic Scaling

As the number of messages inside the queue increases, AWS automatically creates additional Lambda executions to process them.

This enables serverless applications to scale based on demand without requiring manual intervention.

Unlike traditional worker servers, there is no infrastructure that needs to be provisioned or managed.

Learning Through Implementation

Before this week, I associated Lambda almost exclusively with API Gateway. Learning about Event Source Mappings demonstrated that Lambda can also function as a background worker capable of continuously processing events generated by other AWS services.

This significantly expanded my understanding of how serverless applications are designed.

Application in this Project

The Processor Lambda was connected to the SQS queue using a Lambda Event Source Mapping. Whenever S3 generated a new event, Lambda automatically retrieved the corresponding message from the queue and processed the uploaded image without any explicit invocation from the frontend or backend.

AMAZON SIMPLE NOTIFICATION SERVICE (AMAZON SNS)

After understanding how Amazon SQS enables asynchronous processing, the next service introduced was **Amazon Simple Notification Service (SNS)**. While both SQS and SNS deal with message delivery, they solve different problems within a distributed system.

Amazon SNS is a fully managed **publish-subscribe (Pub/Sub)** messaging service. Instead of storing messages for later processing, SNS immediately delivers a published message to one or more subscribed endpoints.

This makes SNS particularly useful for notifications, alerts, and broadcasting events to multiple services simultaneously.

Why AWS Provides SNS

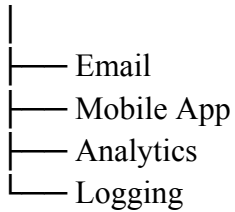
Many applications need to notify multiple systems whenever an important event occurs.

For example, after successfully processing an image, an application might need to:

- Send an email to the user.
- Notify a mobile application.
- Trigger another Lambda function.
- Publish an analytics event.

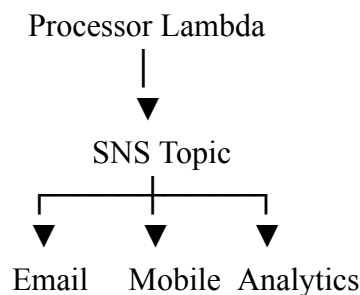
Without SNS, the application would need to invoke each service individually.

Processor Lambda



As the number of consumers increases, the application becomes tightly coupled to every downstream service.

SNS solves this by introducing a **Topic** between the publisher and subscribers.



The publisher only sends a single message to the topic. SNS handles delivering that message to every subscriber.

Topics

A **Topic** acts as the communication channel between publishers and subscribers.

Instead of sending notifications directly to individual services, publishers send messages to a topic.

Subscribers registered with that topic automatically receive the notification.

Publishers

A **Publisher** is any application or AWS service that sends messages to an SNS topic.

In this project, the **Processor Lambda** acted as the publisher after successfully completing image processing.

Subscribers

Subscribers are the endpoints that receive messages published to a topic.

SNS supports multiple subscriber types, including:

- Email
- SMS
- AWS Lambda
- Amazon SQS
- HTTP/HTTPS endpoints

This flexibility allows the same notification to be delivered through different communication channels.

Fan-out Messaging

One of the biggest advantages of SNS is **fan-out messaging**.

A single published message can be delivered to multiple subscribers simultaneously.

This allows new functionality to be added without modifying the publisher.

For example, the application could later support mobile push notifications simply by adding another subscriber to the topic.

Publishing a Notification

Once image processing completed, the Lambda function only needed to publish a message to an SNS topic.

```
sns.publish(  
    TopicArn=TOPIC_ARN,  
    Subject="Image Processed",  
    Message="Your uploaded image has been processed."  
)
```

Notice that the Lambda function does not send emails directly. Its only responsibility is publishing the event. SNS determines which subscribers receive the notification, keeping the processing logic independent from the notification mechanism.

SNS vs Amazon SQS

Although both services transfer messages, their responsibilities are different.

Amazon SQS	Amazon SNS
Stores messages	Broadcasts messages
Queue-based	Publish-Subscribe model
One consumer processes each message	Multiple subscribers receive the same message
Used for asynchronous processing	Used for notifications and event broadcasting

Understanding this distinction helped clarify why both services were required within the same architecture.

Learning Through Implementation

Initially, I assumed the Processor Lambda would directly send email notifications after processing completed.

As the architecture evolved, using Amazon SNS created a much cleaner design. Instead of depending on a specific notification method, the processor simply published a message to an SNS topic, allowing additional subscribers to be added in the future without modifying the processing logic.

This demonstrated another example of how loosely coupled systems are easier to extend.

Application in this Project

After successfully processing an uploaded image, the Processor Lambda published a completion message to an SNS topic. An email subscription attached to the topic then notified the user that processing had finished.

ADVANCED AMAZON COGNITO FEATURES

Amazon Cognito was introduced in Week 2 to provide user authentication and authorization. This week expanded on that foundation by exploring several advanced features that simplify authentication for production applications.

Rather than focusing on user management, the emphasis shifted towards **how authentication integrates with serverless applications**.

Cognito Hosted UI

Instead of developing a custom login page, Amazon Cognito provides a fully managed **Hosted UI** that handles:

- User Sign Up
- User Sign In
- Password Reset
- Email Verification

Users authenticate directly with Cognito, reducing the amount of authentication logic required within the application.

Using the Hosted UI also allows applications to adopt standard OAuth authentication flows without implementing them manually.

OAuth Authorization Code Flow

Another important concept introduced this week was the **OAuth Authorization Code Flow**.

Instead of immediately returning access tokens after authentication, Cognito first returns an **Authorization Code**.

The frontend then exchanges this code for:

- Access Token
- ID Token
- Refresh Token

User Login



Hosted UI



Authorization Code



Token Exchange



JWT Tokens

This approach is more secure than directly exposing tokens during authentication and is commonly used by modern web applications.

JWT Authorizer

In previous weeks, Lambda functions manually extracted JWT claims after receiving requests through API Gateway.

This week introduced the **API Gateway JWT Authorizer**, which validates the JWT before the request reaches the Lambda function.

The authorizer automatically verifies:

- Token signature
- Token expiration
- Token issuer
- Audience

Only valid requests are forwarded to the backend, reducing the amount of authentication logic required within Lambda.

Pre Token Generation Trigger

Another useful Cognito feature introduced this week was the **Pre Token Generation Trigger**.

Before Cognito issues a JWT, it can invoke a Lambda function that modifies the claims included in the token.

This allows applications to:

- Add custom claims.
- Modify existing claims.
- Include application-specific information inside the JWT.

Instead of performing additional database lookups during every request, the required authorization information can be embedded directly into the token.

Learning Through Implementation

Before this week, I viewed Cognito primarily as a service for user registration and login.

Exploring the Hosted UI, OAuth flows, JWT Authorizers, and Pre Token Generation triggers demonstrated that Cognito provides much more than authentication. It also simplifies authorization and reduces the amount of custom authentication logic that applications need to maintain.

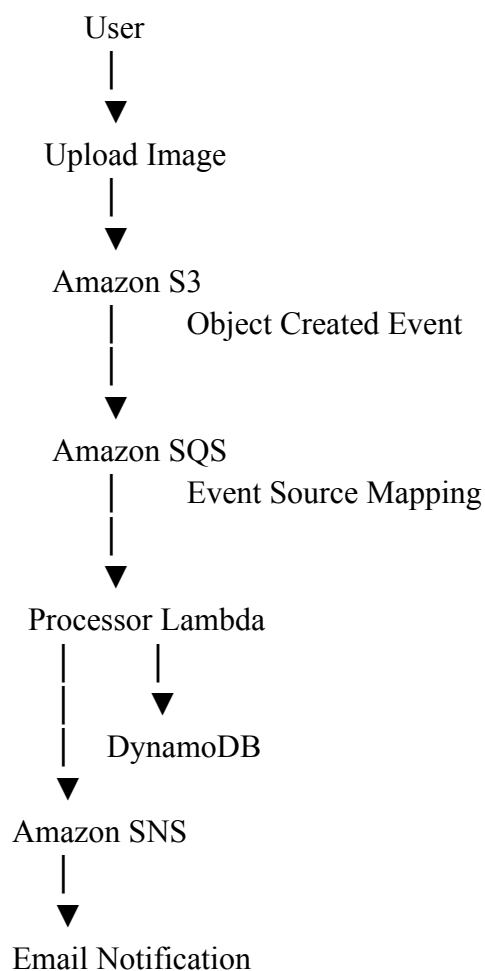
Application in this Project

The application used the Cognito Hosted UI for user authentication, while API Gateway JWT Authorizers validated incoming access tokens before forwarding requests to the backend. A Pre Token Generation trigger was also used to customize JWT claims, allowing user-specific information to be included within the issued tokens.

3. PROJECT IMPLEMENTATION

FINAL ARCHITECTURE

After understanding the individual AWS services, the final implementation combined them into a loosely coupled event-driven pipeline.



Unlike the previous week's project, the frontend no longer coordinated the entire workflow. Its responsibility ended once the image upload completed successfully. Every subsequent step was automatically performed by AWS services through events.

IMAGE UPLOAD WORKFLOW

The workflow began when a user uploaded an image to the S3 bucket.

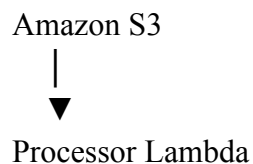
Instead of making another API request to begin processing, the upload itself became the trigger for the remaining pipeline.

This simplified the frontend while reducing unnecessary network requests.

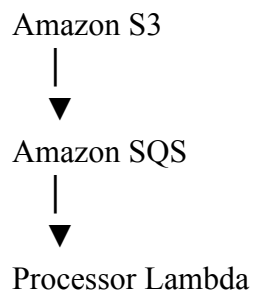
DECOUPLING UPLOADS FROM PROCESSING

One of the most significant architectural decisions during implementation was introducing Amazon SQS between Amazon S3 and the Processor Lambda.

The original idea was to invoke the processing Lambda directly after an upload completed.



While this approach worked, introducing Amazon SQS provided several advantages.



The queue isolated uploads from processing, improved fault tolerance through automatic retries, and allowed processing capacity to scale independently of upload traffic.

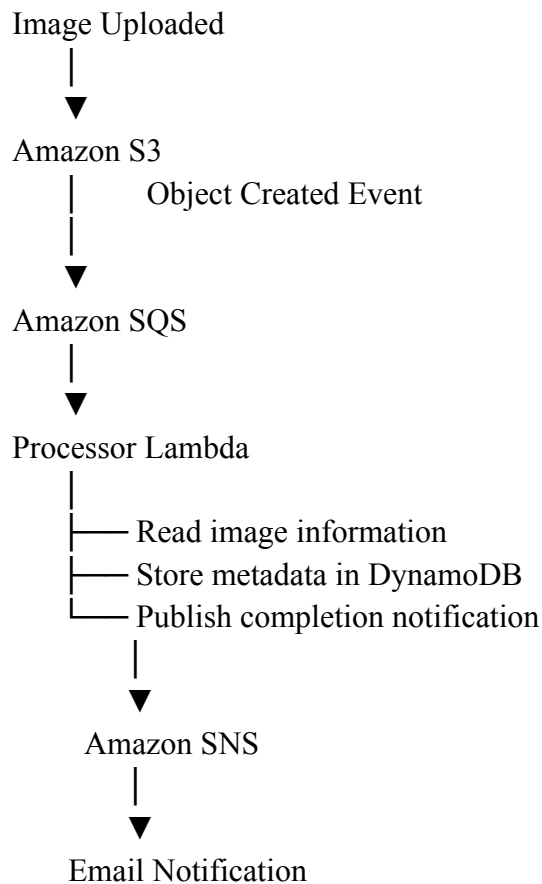
This refinement resulted in a more robust and production-oriented architecture.

PROCESSOR WORKFLOW

Once a message became available in the SQS queue, AWS automatically invoked the Processor Lambda through the Event Source Mapping.

The Lambda function performed the processing required for the uploaded image before storing the processed information and publishing a notification through Amazon SNS.

The complete workflow is shown below.



Each AWS service was responsible for a single stage of the workflow, making the overall architecture easier to understand and extend.

SIMPLIFYING THE PROCESSING PIPELINE

One of the biggest architectural discussions during the week involved deciding **how much work should be performed after an image upload**.

The initial project specification proposed a more complex processing pipeline with additional image processing functionality. As development progressed, the implementation was simplified to focus on building a reliable event-driven architecture rather than image manipulation itself.

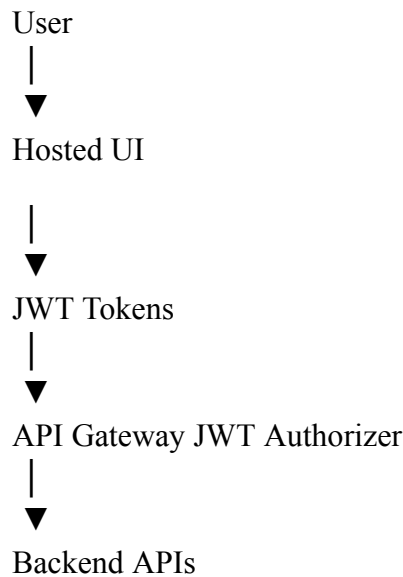
This resulted in a much cleaner workflow where the emphasis shifted from image processing algorithms to understanding how AWS services communicate asynchronously.

The biggest lesson from this refinement was that the primary objective of the project was not image processing, but learning how to design and implement an event-driven serverless application.

AUTHENTICATION FLOW

Although authentication had already been introduced in previous weeks, this project integrated it more naturally into the overall architecture.

The application used Amazon Cognito Hosted UI for user authentication. Once users successfully signed in, Cognito generated JWT tokens using the OAuth Authorization Code Flow. API Gateway validated these tokens using a JWT Authorizer before forwarding requests to the backend.



This approach reduced the amount of authentication logic inside the Lambda functions while ensuring that only authenticated requests reached the backend.

4. KEY LEARNINGS & TAKEAWAYS

Week 4 marked an important shift from building serverless APIs to designing **event-driven cloud applications**. The biggest learning was understanding that not every workflow should be implemented using direct request-response communication. Instead, AWS provides several managed services that allow applications to react to events asynchronously while remaining loosely coupled.

Some of the most important takeaways from this week include:

- Understanding the differences between **request-response** and **event-driven** architectures.
- Learning how **Amazon S3 Event Notifications** automatically initiate serverless workflows.
- Understanding how **Amazon SQS** enables reliable asynchronous communication between services.

- Learning how **Lambda Event Source Mappings** automatically poll queues and invoke Lambda functions without additional application logic.
- Understanding the **Publish-Subscribe** communication model through Amazon SNS.
- Exploring advanced **Amazon Cognito** features, including Hosted UI, OAuth Authorization Code Flow, JWT Authorizers, and Pre Token Generation triggers.
- Appreciating the importance of **loosely coupled architectures** when designing scalable cloud-native applications.
- Recognizing that architecture is often more important than individual services. Selecting the right communication pattern can significantly simplify an application's design.

Compared to the previous weeks, this project focused less on individual AWS services and more on how those services work together to build reliable distributed systems. It demonstrated that serverless applications extend beyond APIs and databases, relying on events and managed messaging services to coordinate complex workflows automatically.